

MODUL IV

FUNCTION

4.1 TUJUAN PRAKTIKUM

Tujuan Praktikum pada Modul IV *Function* adalah:

1. Mahasiswa mengenal fungsi dalam bahasa C++.
2. Mahasiswa mampu mengimplementasikan jenis-jenis fungsi dalam pembuatan program.
3. Mahasiswa memahami tujuan penggunaan fungsi dalam program.

4.2 DASAR TEORI

4.2.1 Pengertian *Function*

Fungsi atau *function* dalam bahasa C++ merupakan sebuah blok kode yang berdiri sendiri dan dapat dipanggil dari bagian mana pun dalam program. *Function* membantu dalam mengorganisir dan memecah program menjadi bagian-bagian yang lebih kecil, meningkatkan keterbacaan dan pemeliharaan kode [1].

Sebelum sebuah *function* yang digunakan didalam program, biasanya dideklarasikan terlebih dahulu. Deklarasi *function* menyertakan tipe data pengembalian (jika ada), nama *function*, dan daftar parameter yang diterima oleh *function* tersebut. Hal ini penting agar *compiler* dapat mengenali dan memvalidasi pemanggilan *function* sebelum definisi lengkapnya ditemukan [1].

Setelah deklarasi, *function* harus didefinisikan di suatu tempat dalam program. Definisi *function* mencakup blok kode yang menjelaskan tugas yang dilakukan oleh *function* tersebut. Ini mencakup logika, operasi, dan alur kontrol yang terkait dengan *function* tersebut. *Function* dapat dipanggil berkali-kali dari bagian manapun dalam program dengan menyebutkan namanya [2]. Selain itu, penggunaan *function* juga bertujuan untuk memecah program menjadi bagian-bagian yang lebih kecil dan terstruktur, sehingga kode menjadi lebih mudah dipahami, dikelola, dan digunakan kembali. Bentuk umum deklarasi dan definisi *function* pada C++ adalah sebagai berikut:

```
//Deklarasi function
tipe_kembalian nama_function (tipe_parameter nama_parameter);

// definisi function
tipe_kembalian nama_function (tipe_parameter nama_parameter){
    isi function;
    return nilai;
}
```

Berdasarkan bentuk deklarasi *function* tersebut, terdapat beberapa bagian penting yang memiliki fungsi masing-masing, yaitu:

1. “*tipe_kembalian*” digunakan untuk menentukan jenis tipe data nilai yang akan dikembalikan oleh *function*, seperti *int*, *float*, *char* maupun *void* jika *function* tidak mengembalikan nilai apapun.
2. “*nama_function*” digunakan sebagai identitas atau nama *function* yang akan dipanggil dalam program.
3. “*tipe_parameter*” digunakan untuk menentukan jenis tipe data dari parameter yang diterima oleh *function*.
4. “*nama_parameter*” digunakan sebagai identitas atau nama variabel parameter yang akan digunakan di dalam blok kode *function*.
5. “*isi function*” merupakan blok kode yang berisi logika dan operasi yang akan dijalankan ketika *function* dipanggil.
6. “*return nilai*” digunakan untuk mengembalikan nilai hasil eksekusi *function* ke pemanggilnya sesuai dengan tipe kembalian yang dideklarasikan.

Jika sebuah *function* memerlukan *input* untuk menjalankan tugasnya, parameter dapat didefinisikan. Parameter adalah variabel yang digunakan dalam *function* untuk menerima nilai dari pemanggil *function*. Parameter dapat digunakan di dalam blok kode *function* untuk melakukan operasi yang sesuai. Argumen, di sisi lain, adalah nilai yang diberikan saat memanggil *function* dan sesuai dengan parameter yang dideklarasikan oleh fungsi tersebut [2].

4.2.1 Jenis-jenis *Function*

Fungsi atau *function* dalam bahasa C++ memiliki beberapa jenis yang umum digunakan, antara lain:

1. Fungsi Tanpa Pengembalian Nilai (*Function Void*)

Function void adalah fungsi yang tidak mengembalikan nilai apapun ke pemanggilnya. Fungsi ini hanya melakukan suatu proses tertentu seperti mencetak *output* ke layar tanpa menghasilkan nilai kembalian. Hal ini menjadikan *function void* sangat berguna untuk tugas-tugas yang bersifat prosedural. Fungsi *void* cocok digunakan ketika program hanya membutuhkan eksekusi perintah tanpa perlu mendapatkan hasil dari fungsi tersebut [3]. Bentuk umum *function void*:

```
void nama_function(tipe_parameter nama_parameter){  
    isi function;  
}
```

2. Fungsi Tanpa Pengembalian Nilai (*Function Return Value*)

Function Return Value adalah fungsi yang mengembalikan nilai ke pemanggilnya setelah selesai dieksekusi. Tipe nilai yang dikembalikan harus sesuai dengan tipe kembalian yang dideklarasikan pada fungsi. Nilai kembalian dapat berupa tipe data apapun seperti *int*, *float*, *char*, *string*, maupun *bool* [3]. Bentuk umum *function return value*:

```
tipe_kembalian nama_function(tipe_parameter nama_parameter){  
    isi function;  
    return nilai;  
}
```

3. *Function Inline*

Inline Function adalah fungsi yang disarankan kepada *compiler* untuk disisipkan langsung di tempat pemanggilan fungsi menggunakan kata kunci *inline*. Tujuannya adalah mengurangi *overhead* yang terjadi akibat proses pemanggilan fungsi biasa sehingga eksekusi program menjadi lebih cepat. Fungsi *inline* cocok digunakan untuk fungsi yang berukuran kecil dan sering dipanggil [3]. Bentuk umum *function inline*:

```
inline tipe_kembalian nama_function(tipe_parameter nama_parameter){  
    return nilai;  
}
```

4. *Function Overloading*

Function overloading adalah kemampuan mendefinisikan dua atau lebih fungsi dengan nama yang sama namun berbeda parameter baik dari segi jumlah maupun tipe datanya. *Compiler* akan secara otomatis memilih fungsi yang sesuai berdasarkan argumen yang diberikan saat pemanggilan [3]. Bentuk umum *function overloading*:

```
tipe_kembalian nama_function(tipe_parameter1 nama_parameter1)  
tipe_kembalian nama_function(tipe_parameter2 nama_parameter2);
```

5. Fungsi Rekursif (*Function Recursive*)

Function recursive adalah fungsi yang memanggil dirinya sendiri di dalam tubuh fungsi tersebut. Setiap fungsi rekursif wajib memiliki dua bagian yaitu *base case* sebagai kondisi berhenti agar tidak terjadi perulangan tak terbatas, dan *recursive case* sebagai bagian yang memanggil dirinya sendiri dengan parameter yang berubah mendekati *base case* [4]. Bentuk umum *function recursive*:

```
tipe_kembalian nama_function (parameter){  
  
    if (base_case) return nilai;  
    Return nama_function;  
}
```

4.2.2 Jenis Variabel dalam *Function*

Dalam penggunaan *function*, terdapat dua jenis variabel berdasarkan ruang lingkup atau *scope*-nya, yaitu variabel lokal dan variabel global [3].

Variabel lokal yaitu variabel yang dideklarasikan di dalam sebuah *function* dan hanya dapat diakses di dalam *function* tempat variabel tersebut dideklarasikan. Variabel ini bersifat sementara dan akan hilang dari memori ketika *function* selesai dieksekusi sehingga tidak dapat diakses oleh *function* lain meskipun memiliki nama yang sama [3].

Variabel global yaitu variabel yang dideklarasikan di luar semua *function* dan dapat diakses oleh seluruh *function* dalam program. Variabel ini bersifat permanen selama program berjalan dan nilainya dapat diubah oleh *function* manapun yang mengaksesnya sehingga perlu digunakan dengan hati-hati agar tidak menyebabkan kesalahan yang tidak terduga [4]. Bentuk umum variabel lokal dan variabel global adalah sebagai berikut:

```
#include <iostream>
using namespace std;

tipe_data variabel_global;

fungsi nama_function() {
    tipe_data variabel_lokal;
}

int main() {
    return 0;
}
```

4.3 PERMASALAHAN

Setelah menyelesaikan semester yang melelahkan, Tio, Avaro, dan Izzul memutuskan untuk membuka sebuah toko kecil di dekat kos mereka. Dengan semangat yang sama dan modal yang dikumpulkan bersama, mereka mendirikan toko bernama TIVAIZ.

Pada hari pertama membuka toko, Tio, Avaro, dan Izzul menyadari belum ada sistem pencatatan yang teratur. Stok masih dihitung manual, harga ditulis di kertas, dan total nilai barang belum diketahui. Karena itu, mereka sepakat membuat sistem sendiri.

Izzul bertugas mencatat barang masuk, Avaro menghapus barang yang tidak dijual, dan Tio mengubah data harga serta stok. Tio juga membawa kembali kalkulator sederhana yang pernah ia buat pada mata kuliah Sistem Digital dan mengembangkannya ulang menggunakan bahasa C++ dengan konsep function yang baru dipelajari.

```

=====
[          MODULE 4 - FUNCTION          ]
=====
1. TIVAIZ Store
2. Kalkulator TIVAIZ
0. Keluar
=====
Pilih menu: |

```

Gambar 4.1 Tampilan Menu Utama TIVAIZ Store.

Menu 1 TIVAIZ Store memiliki empat sub-menu, yaitu input barang, hapus barang, edit barang, dan lihat barang, serta opsi untuk kembali ke menu sebelumnya.

```

#=====#
|          TIVAIZ STORE          |
#=====#
1. Input Barang
2. Hapus Barang
3. Edit Barang
4. Lihat Barang
0. Kembali
=====
Pilih menu:

```

Gambar 4.1 Tampilan Menu 1 TIVAIZ Store.

Jika input bukan angka atau melebihi jumlah sub-menu, sistem akan menampilkan pesan error.

```
#=====#
|          TIVAIZ STORE          |
#=====#
1. Input Barang
2. Hapus Barang
3. Edit Barang
4. Lihat Barang
0. Kembali
=====
Pilih menu: Input Barang
[Gagal] Input harus berupa angka!
Press any key to continue . . .
```

Gambar 4.3 Tampilan Proses *Error Handling* pada Sub-Menu.

Sub-menu 1 input barang, Izzul memasukkan barang yang akan dijual. Jika input benar, data barang akan disimpan ke array dan jumlah barang otomatis bertambah, lalu sistem kembali ke Menu 1. Jika input salah, sistem akan menampilkan pesan error dan kembali ke Menu 1.

```
#=====#
|          TIVAIZ STORE          |
#=====#

          INPUT BARANG
=====
Nama Barang : Indomie Goreng
=====
Stok (unit) : 10
=====
Harga (Rp)  : 3500
=====
[Berhasil] Indomie Goreng ditambahkan! (Total barang: 1)
Press any key to continue . . .
```

Gambar 4.4 Tampilan Sub-Menu 1 *Input Barang*.

Sub-menu 2 hapus barang, Avaro menghapus barang yang tidak dijual. Data dihapus dari array, elemen digeser, dan total barang berkurang. Jika input salah, sistem akan menampilkan pesan error dan kembali ke Menu 1.

```
#=====#
|          TIVAIZ STORE          |
#=====#

          HAPUS BARANG
=====
No       : 1
Nama Barang : Indomie Goreng
Stok Barang : 10
Harga/Stok : Rp 3500
=====
No       : 2
Nama Barang : Meja Bundar
Stok Barang : 5
Harga/Stok : Rp 100000
=====
No       : 3
Nama Barang : Rubik 3x3
Stok Barang : 10
Harga/Stok : Rp 35000
=====
Pilih nomor barang yang dihapus (1-3): 2
=====
[Berhasil] Meja Bundar dihapus! (Total barang: 2)
Press any key to continue . . .
```

Gambar 4.5 Tampilan Sub-Menu 2 *Hapus Barang*.

Sub-menu 3 edit barang, Tio mengubah data harga serta stok barang. Stok bisa ditambah atau dikurangi, tetapi tidak boleh menjadi nol. Penghapusan barang dilakukan melalui menu hapus. Jika input salah, sistem akan menampilkan pesan error dan kembali ke Menu 1.

```
#=====#
|      TIVAIZ STORE      |
#=====#

EDIT BARANG

Barang: Rubik 3x3

1. Edit Harga
2. Tambah Stok
3. Kurangi Stok
0. Kembali

Pilih opsi: 2

Jumlah tambah: 3

[Berhasil] Stok Rubik 3x3: 10 -> 13
Press any key to continue . . .
```

Gambar 4.6 Tampilan Sub-Menu 3 Edit Barang.

Sub-menu 4 lihat barang, setiap sore sebelum tutup, mereka mengecek kondisi toko bersama. Setelah pengecekan selesai, sistem kembali ke Menu 1.

```
#=====#
|      TIVAIZ STORE      |
#=====#

LIHAT BARANG

No       : 1
Nama Barang : Indomie Goreng
Stok Barang : 10
Harga/Stok : Rp 3500
Total Harga : Rp 35000

No       : 2
Nama Barang : Rubik 3x3
Stok Barang : 13
Harga/Stok : Rp 35000
Total Harga : Rp 455000

Total barang : 2 jenis

Press any key to continue . . .
```

Gambar 4.7 Tampilan Sub-Menu 4 Lihat Barang.

Menu 2 Kalkulator TIVAIZ S terdapat dua sub-menu. Jika input bukan huruf atau melebihi jumlah sub-menu, sistem akan menampilkan pesan error.

```
+=====+
|      K4LKUL4T0R      |
X=====X/

1. Operasi Dasar (+, -, *, /)
2. Faktorial
0. Kembali

=====
Pilih menu:
```

Gambar 4.8 Tampilan Menu 2 Kalkulator TIVAIZ.

Sub-menu 1 operasi dasar, lakukan operasi penjumlahan, pengurangan, perkalian, dan pembagian berdasarkan input operator terlebih dahulu, lalu dua angka. Pembagian dengan nol tidak bisa dilakukan.

```

+=====+
|          K4LKUL4T0R          |
+=====+
Operator (+, -, *, /) : +
Angka pertama : 10
Angka kedua   : 20
10 + 20 = 30
Press any key to continue . . .

```

Gambar 4.9 Tampilan Sub-Menu 1 Operasi Dasar.

Sub-menu 2 faktorial, hitung faktorial menggunakan rumus rekursif $n! = n \times (n-1)!$ dengan base case $0! = 1$. Wajib menggunakan fungsi rekursif tanpa looping. Angka negatif tidak valid.

```

+=====+
|          K4LKUL4T0R          |
+=====+
N : 10
10! = 3628800
Press any key to continue . . .

```

Gambar 4.10 Tampilan Sub-Menu 2 Faktorial.

Program akan berhenti ketika user input angka 0 pada menu utama dan akan menampilkan pesan perpisahan.

```

Sampai jumpa! Terima kasih telah menggunakan Toko TIVAIZ!
=====

```

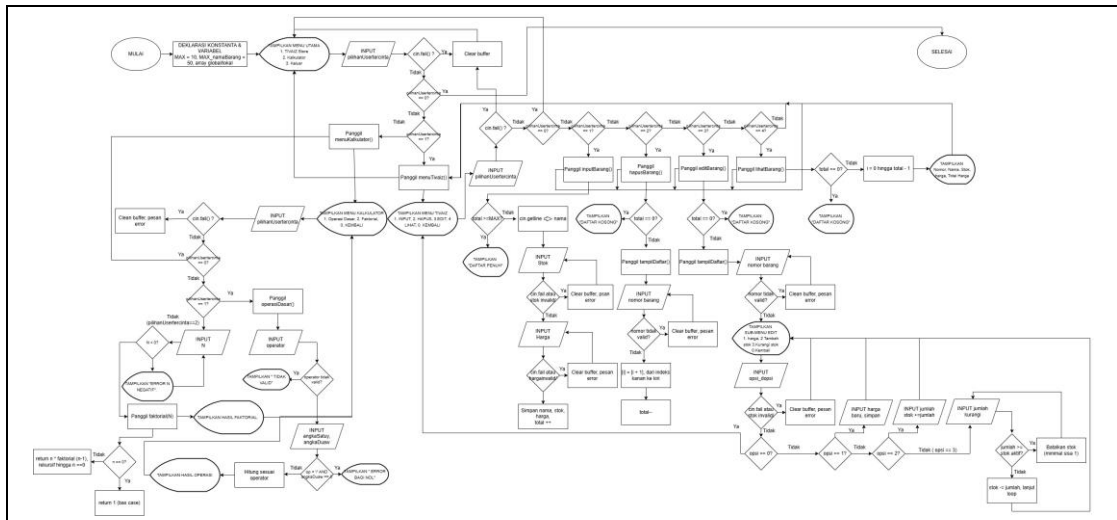
Gambar 4.10 Tampilan Pesan Perpisahan

4.4 HASIL PERCOBAAN

1. Algoritma

- a. Mulai.
- b. Deklarasi konstanta $MAX = 10$, $MAX_namaBarang = 50$, serta *array* global/lokal: *namaBarang*, *stokBarang*, *hargaBarang*, dan *totalBarang* = 0.
- c. Aturan Validasi Global: Setiap *input* menu/angka yang gagal (*cin.fail()*) atau nomor di luar jangkauan akan otomatis membersihkan *buffer*, menampilkan pesan error, lalu mengulang menu atau keluar fungsi.
- d. Fungsi *menuUtama()*: Tampilkan menu utama (1. TIVAIZ Store, 2. Kalkulator TIVAIZ, 0. Keluar). *Input* pilihan; panggil *menuTivaiz()* jika pilih 1, panggil *menuKalkulator()* jika pilih 2. Ulangi hingga pilih 0 (Selesai).
- e. Fungsi *menuTivaiz()*: Tampilkan pilihan (1. Input, 2. Hapus, 3. Edit, 4. Lihat, 0. Kembali). Arahkan ke fungsi terkait sesuai *input* pengguna. Ulangi hingga pilih 0.
- f. Fungsi *inputBarang()*: Jika $total \geq MAX$ tampilkan pesan penuh lalu keluar. Jika aman, *input* nama (*cin.getline*), *stok*, dan *harga*. Jika semua *input* valid, $total++$.
- g. Fungsi *hapusBarang()*: Jika $total == 0$ tampilkan pesan kosong lalu keluar. Jika ada, panggil *tampilDaftar()*, *input* nomor barang, lalu geser seluruh elemen *array* di kanan indeks tersebut ke kiri sebanyak satu slot, kemudian $total--$.
- h. Fungsi *editBarang()*: Jika $total == 0$ keluar. Jika ada, panggil *tampilDaftar()* dan *input* nomor barang. Tampilkan sub-menu edit (1. Harga, 2. Tambah Stok, 3. Kurangi Stok, 0. Kembali), lalu panggil fungsi edit yang sesuai hingga opsi = 0.
- i. Fungsi Aksi Edit:
editHarga: *Input* dan timpa harga lama dengan harga baru.
tambahStok: *Input* jumlah, lalu $stok += jumlah$.
kurangiStok: *Input* jumlah; jika $jumlah \geq stok$ aktif maka batalkan (*stok* minimal sisa 1), jika aman lakukan $stok -= jumlah$.
- j. Fungsi *lihatBarang()*: Jika $total == 0$ tampilkan pesan kosong. Jika ada, lakukan perulangan cetak data No, Nama, Stok, Harga, dan kalkulasi Total Harga ($stok * harga$) dari indeks 0 hingga $total - 1$.
- k. Fungsi *menuKalkulator()*: Tampilkan pilihan (1. Operasi Dasar, 2. Faktorial, 0. Kembali). Jika pilih 1 panggil *operasiDasar()*; jika pilih 2 *input* N (jika $N \geq 0$ panggil rekursif *faktorial(N)*, jika negatif tampilkan error). Ulangi hingga pilih 0.

1. Fungsi operasiDasar(): *Input* karakter operator (+, -, *, /). Jika tidak valid, keluar. *Input* angkaSatuy dan angkaDuaw, lalu hitung sesuai operator. Khusus pembagian (/): jika angkaDuaw == 0 tampilkan error pembagian nol.
 - m. Fungsi Rekursif faktorial(n): Jika n == 0 kembalikan nilai 1 (*base case*), jika tidak kembalikan nilai n * faktorial(n - 1).
 - n. Selesai.
2. *Flowchart*



Gambar 3.8 Flowchart Program TIVAIZ Store.

3. Source Code

```
#include <iostream>
using namespace std;
const int MAX = 10;
const int MAX_namaBarang = 50;
void menuUtama(char nama[][MAX_namaBarang], int stok[], int harga[], int &total);
void menuTivaiz(char nama[][MAX_namaBarang], int stok[], int harga[], int &total);
void inputBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int &total);
void hapusBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int &total);
void editBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int total);
void lihatBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int total);
void editHarga(int &harga, char namaBarang[]);
void tambahStok(int &stok, char namaBarang[]);
void kurangiStok(int &stok, char namaBarang[]);
void menuKalkulator();
void operasiDasar();
```

```

long long faktorial(int n);
void bannerTivaiz (){
    cout << "#=====#" << endl;
    cout << "|                TIVAIZ STORE                |" << endl;
    cout << "#=====#" << endl
<< endl;
}
void bannerKalkulator (){
    cout << "#=====#" << endl;
    cout << "|                KALKULATOR                |" << endl;
    cout << "#=====#" << endl
<<endl;
}
void tampilDaftar(char nama[][MAX_namaBarang], int stok[], int harga[],
int total) {
    for (int i = 0; i < total; i++) {
        cout << "No          : " << i + 1 << "\n";
        cout << "Nama Barang : " << nama[i] << "\n";
        cout << "Stok Barang : " << stok[i] << "\n";
        cout << "Harga/Stok  : Rp " << harga[i] << "\n";
        cout << "-----\n";
    }
}
void menuUtama(char nama[][MAX_namaBarang], int stok[], int harga[], int
&total) {
    int pilihanUsertercinta;
    do{
        cout << "|=====|" <<
endl;
        cout << "|                |" <<
endl;
        cout << "[                MODULE 4 - FUNCTION                ]" <<
endl;
        cout << "|                |" <<
endl;
        cout << "|=====|" <<
endl << endl;
        cout << "1. TIVAIZ Store" << endl << endl;
        cout << "2. Kalkulator TIVAIZ" << endl << endl;
        cout << "0. Keluar" << endl << endl;
        cout << "===== " << endl;
        cout << "Pilihan menu: ";
        cin >> pilihanUsertercinta;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";

```

```

        pilihanUsertercinta = -1;
        system ("pause");
        system ("cls");
        continue;
    }
    if (pilihanUsertercinta == 1)
        menuTivaiz (nama, stok, harga, total);
    else if (pilihanUsertercinta == 2)
        menuKalkulator();
    else if (pilihanUsertercinta == 0)
        cout << "Terima kasih telah menggunakan TIVAIZ Store";
    else{
        cout << "[Gagal] Menu tidak valid! Pilih antara 0 sampai
2.\n";
        system ("pause");
        system ("cls");
    }
    }while (pilihanUsertercinta != 0);
}
void menuTivaiz (char nama[][MAX_namaBarang], int stok[], int harga[],
int &total){
    int pilihanUsertercinta;
    do{
        system ("cls");
        bannerTivaiz();
        cout << "1. Input Barang\n" << endl;
        cout << "2. Hapus Barang\n" << endl;
        cout << "3. Edit Barang\n" << endl;
        cout << "4. Lihat Barang\n" << endl;
        cout << "0. Kembali\n" << endl;
        cout << "=====\n";
        cout << "Pilih menu: ";
        cin >> pilihanUsertercinta;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";
            pilihanUsertercinta = -1;
            system ("pause");
            system ("cls");
            continue;
        }
        if (pilihanUsertercinta== 1)
            inputBarang(nama, stok, harga, total);
        else if (pilihanUsertercinta == 2)
            hapusBarang(nama, stok, harga, total);
        else if (pilihanUsertercinta == 3)
            editBarang (nama, stok, harga, total);
    }
}

```

```

        else if (pilihanUsertercinta == 4)
            lihatBarang(nama, stok, harga, total);
        else if (pilihanUsertercinta == 0) {
            system ("cls");
        }
        else {
            cout << "[Gagal] Menu tidak valid! Pilih antara 0 sampai
4.\n";
            system ("pause");
            system ("cls");
        }
    }while (pilihanUsertercinta != 0);
}

void inputBarang (char nama[][MAX_namaBarang], int stok[], int harga[],
int &total){
    system ("cls");
    bannerTivaiz ();
    cout << "-----" << endl;
    cout << "                INPUT BARANG                " << endl
<< endl;
    cout << "-----" << endl;
    if (total >= MAX){
        cout << "[Gagal] Kapasitas barang penuh!\n";
        system ("pause");
        return;
    }
    cout << "Nama Barang   : ";
    cin.ignore();
    cin.getline(nama[total], MAX_namaBarang);
    cout << "-----\n";
    cout << "Stok (unit)   : ";
    cin >> stok[total];
    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Stok harus berupa angka!\n";
        system ("pause");
        return;
    }
    cout << "-----\n";
    cout << "Harga (Rp)    : ";
    cin >> harga[total];
    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Harga harus berupa angka!\n";
        system ("pause");
        return;
    }
}

```

```

    }
    total++;
    cout << "\n[Berhasil] " << nama[total - 1]
        << " ditambahkan! (Total barang: " << total << ")\n";
    system("pause");
}

void hapusBarang(char nama[][MAX_namaBarang], int stok[], int harga[],
int &total) {
    system("cls");
    bannerTivaiz();
    cout << "-----" << endl;
    cout << "                HAPUS BARANG                " << endl
<< endl;
    cout << "-----" << endl;
    if (total == 0){
        cout << "[Gagal] Belum ada barang!\n";
        system("pause");
        return;
    }
    tampilDaftar(nama, stok, harga, total);
    cout << "Pilih nomor barang yang dihapus (1-" << total << "): ";
    int pilihanUsertercinta;
    cin >> pilihanUsertercinta;

    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";
        system("pause");
        return;
    }
    if (pilihanUsertercinta < 1 || pilihanUsertercinta > total) {
        cout << "[Gagal] Nomor " << pilihanUsertercinta << " tidak
valid!"
        << " Pilih antara 1 sampai " << total << ".\n";
        system("pause");
        return;
    }
    char namaHapus[MAX_namaBarang];
    for (int k = 0; k < MAX_namaBarang; k++) namaHapus[k] =
nama[pilihanUsertercinta - 1][k];

    for (int i = pilihanUsertercinta - 1; i < total - 1; i++) {
        for (int k = 0; k < MAX_namaBarang; k++) nama[i][k] = nama[i +
1][k];
        stok[i] = stok[i + 1];
        harga[i] = harga[i + 1];
    }
}

```

```

        total--;
        cout << "[Berhasil] " << namaHapus
            << " dihapus! (Total barang: " << total << ")\n";
        system ("pause");
    }
    void editHarga(int &harga, char namaBarang[]) {
        int hargaBaru;
        cout << "-----\n";
        cout << "Harga baru (Rp): ";
        cin >> hargaBaru;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";
            system ("pause");
            return;
        }
        int hargaLama = harga;
        harga = hargaBaru;
        cout << "[Berhasil] Harga " << namaBarang
            << ": Rp " << hargaLama << " -> Rp " << harga << "\n";
        system ("pause");
    }
    void tambahStok(int &stok, char namaBarang[]) {
        int jumlah;
        cout << "-----\n";
        cout << "Jumlah tambah: ";
        cin >> jumlah;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";
            system ("pause");
            return;
        }
        int stokLama = stok;
        stok += jumlah;
        cout << "[Berhasil] Stok " << namaBarang
            << ": " << stokLama << " -> " << stok << "\n";
        system ("pause");
    }
    void kurangiStok(int &stok, char namaBarang[]) {
        int jumlah;
        cout << "-----\n";
        cout << "Jumlah kurang: ";
        cin >> jumlah;
        if (cin.fail()) {
            cin.clear();

```

```

        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";
        system("pause");
        return;
    }
    if (jumlah >= stok) {
        cout << "[Gagal] Stok minimal 1!\n";
        cout << "Stok saat ini : " << stok << "\n";
        cout << "Maks dikurangi : " << stok - 1 << "\n";
        system("pause");
        return;
    }
    int stokLama = stok;
    stok -= jumlah;
    cout << "[Berhasil] Stok " << namaBarang
        << ": " << stokLama << " -> " << stok << "\n";
    system("pause");
}

void editBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int
total) {
    system("cls");
    bannerTivaiz();
    cout << "-----" << endl;
    cout << "                EDIT BARANG                " << endl
<< endl;
    cout << "-----" << endl;
    if (total == 0) {
        cout << "[Gagal] Belum ada barang!\n";
        system("pause");
        return;
    }
    tampilDaftar(nama, stok, harga, total);
    cout << "Pilih nomor barang yang diedit (1-" << total << "): ";
    int pilihanUsertercinta;
    cin >> pilihanUsertercinta;
    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";
        system("pause");
        return;
    }
    if (pilihanUsertercinta < 1 || pilihanUsertercinta > total) {
        cout << "[Gagal] Nomor " << pilihanUsertercinta << " tidak
valid!"
        << " Pilih antara 1 sampai " << total << ".\n";
        system("pause");
        return;
    }
}

```



```

    }
    int indeks = pilihanUsertercinta - 1;
    int opsi_dopsi;
    do {
        system ("cls");
        bannerTivaiz();
        cout << "-----" <<
endl;
        cout << "                EDIT BARANG                " <<
endl << endl;
        cout << "-----" <<
endl;
        cout << "Barang: " << nama[indeks] << "\n";
        cout << "-----\n";
        cout << "1. Edit Harga\n" << endl;
        cout << "2. Tambah Stok\n" << endl;
        cout << "3. Kurangi Stok\n" << endl;
        cout << "0. Kembali\n" << endl;
        cout << "-----\n";
        cout << "Pilih opsi: ";
        cin >> opsi_dopsi;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";
            system ("pause");
            continue;
        }
        if (opsi_dopsi == 1)
            editHarga (harga[indeks], nama[indeks]);
        else if (opsi_dopsi == 2)
            tambahStok (stok[indeks], nama[indeks]);
        else if (opsi_dopsi == 3)
            kurangiStok(stok[indeks], nama[indeks]);
        else if (opsi_dopsi == 0) {}
        else {
            cout << "[Gagal] Opsi tidak valid!\n";
            system ("pause");
        }
    } while (opsi_dopsi != 0);
}

void lihatBarang(char nama[][MAX_namaBarang], int stok[], int harga[],
int total) {
    system ("cls");
    bannerTivaiz();
    cout << "-----" <<
endl;

```

```

        cout << "                                LIHAT BARANG                                " <<
endl << endl;
        cout << "-----" <<
endl;
        if (total == 0) {
            cout << "Belum ada barang di toko.\n";
        } else {
            for (int i = 0; i < total; i++) {
                cout << "No          : " << i + 1 << "\n";
                cout << "Nama Barang : " << nama[i] << "\n";
                cout << "Stok Barang : " << stok[i] << "\n";
                cout << "Harga/Stok  : Rp " << harga[i] << "\n";
                cout << "Total Harga : Rp " << stok[i] * harga[i] << "\n";
                cout << "-----\n";
            }
            cout << "Total barang : " << total << " jenis\n";
        }
        cout << "-----\n";
        system ("pause");
    }
    void menuKalkulator() {
        int pilihanUsertercinta;
        do {
            system ("cls");
            bannerKalkulator();
            cout << "1. Operasi Dasar (+, -, *, /)\n" << endl;
            cout << "2. Faktorial\n" << endl;
            cout << "0. Kembali\n" << endl;
            cout << "=====\n";
            cout << "Pilih menu: ";
            cin >> pilihanUsertercinta;
            if (cin.fail()) {
                cin.clear();
                cin.ignore(1000, '\n');
                cout << "[Gagal] Input harus berupa angka!\n";
                pilihanUsertercinta = -1;
                system ("pause");
                system ("cls");
                continue;
            }
            if (pilihanUsertercinta == 1) {
                operasiDasar();
            } else if (pilihanUsertercinta == 2) {
                system ("cls");
                bannerKalkulator();
                cout << "-----\n";
                int nakHitung_apa;
                cout << "N : ";
            }
        } while (pilihanUsertercinta != 0);
    }
}

```

```

        cin >> nakHitung_apa;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";
            system ("pause");
            continue;
        }
        cout << "-----\n";
        if (nakHitung_apa < 0) {
            cout << "[Gagal] Faktorial tidak bisa untuk angka
negatif!\n";
        } else {
            cout << nakHitung_apa << "! = " <<
faktorial(nakHitung_apa) << "\n";
        }
        system ("pause");
    } else if (pilihanUsertercinta == 0) { //balik
        system ("cls");
    } else {
        cout << "[Gagal] Menu tidak valid! Pilih antara 0 sampai
2.\n";
        system ("pause");
        system ("cls");
    }
} while (pilihanUsertercinta != 0);
}

void operasiDasar() {
    system ("cls");
    bannerKalkulator();
    cout << "-----\n";
    char ini_operator;
    cout << "Operator (+, -, *, /) : ";
    cin >> ini_operator;
    if (ini_operator != '+' && ini_operator != '-' && ini_operator != '*'
&& ini_operator != '/') {
        cout << "[Gagal] Operator tidak valid!\n";
        system ("pause");
        return;
    }
    float angkaSatuy, angkaDuaw;
    cout << "-----\n";
    cout << "Angka pertama : ";
    cin >> angkaSatuy;
    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";

```

```

        system ("pause");
        return;
    }
    cout << "-----\n";
    cout << "Angka kedua    : ";
    cin >> angkaDuaw;
    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";
        system ("pause");
        return;
    }
    cout << "-----\n";
    if      (ini_operator == '+') cout << angkaSatuy << " + " <<
angkaDuaw << " = " << angkaSatuy + angkaDuaw << "\n";
    else if (ini_operator == '-') cout << angkaSatuy << " - " <<
angkaDuaw << " = " << angkaSatuy - angkaDuaw << "\n";
    else if (ini_operator == '*') cout << angkaSatuy << " * " <<
angkaDuaw << " = " << angkaSatuy * angkaDuaw << "\n";
    else if (ini_operator == '/') {
        if (angkaDuaw == 0) cout << "[Gagal] Tidak bisa dibagi nol!\n";
        else                cout << angkaSatuy << " / " << angkaDuaw << " = " <<
angkaSatuy / angkaDuaw << "\n";
    }
    system ("pause");
}
long long faktorial(int n) {
    if (n == 0) return 1;
    return n * faktorial(n - 1);
}
int main (){
    char namaBarang[MAX][MAX_namaBarang];
    int  stokBarang[MAX];
    int  hargaBarang[MAX];
    int  totalBarang = 0;
    menuUtama(namaBarang, stokBarang, hargaBarang, totalBarang);
    return 0;
}

```

4. Hasil Program

```

=====
|                                     |
|             MODULE 4 - FUNCTION     |
|                                     |
|=====|
1. TIVAIZ Store
2. Kalkulator TIVAIZ
0. Keluar

=====
Pilihan menu:

```

Gambar 4.11 Tampilan Menu Utama TIVAIZ Store.

Berdasarkan **Gambar 4.11** di atas, dapat diketahui bahwa ketika program pertama kali dijalankan, sistem akan mengeksekusi fungsi “menuUtama()” untuk menyajikan antarmuka pengguna tingkat awal. Fungsi ini menampilkan tiga opsi navigasi utama bagi pengguna, yaitu menu “1. TIVAIZ Store” untuk manajemen inventaris toko, “2. Kalkulator TIVAIZ” untuk komputasi aritmatika dan faktorial, serta menu “0. Keluar” untuk menghentikan operasional sistem. *Input* pilihan menu dari pengguna akan ditampung ke dalam variabel “pilihanUsertercinta” melalui aliran *stream* cin untuk kemudian divalidasi ke dalam struktur percabangan sebelum mengarahkan alur kendali program ke fungsi sub-menu berikutnya.

```

#####
|             TIVAIZ STORE             |
|#####|
1. Input Barang
2. Hapus Barang
3. Edit Barang
4. Lihat Barang
0. Kembali

=====
Pilih menu: |

```

Gambar 4.12 Tampilan Sub-Menu TIVAIZ Store.

Berdasarkan **Gambar 4.12** di atas, dapat diketahui bahwa ketika pengguna memilih opsi “1” pada menu utama, program akan mengalihkan alur kendali sistem untuk mengeksekusi fungsi “menuTivaiz()”. Fungsi ini menampilkan antarmuka sub-menu khusus manajemen inventaris toko dengan lima pilihan instruksi operasional data, meliputi “1. Input Barang”, “2. Hapus Barang”, “3. Edit Barang”, “4. Lihat Barang”, serta “0. Kembali”. Sistem memanfaatkan fungsi “system(“cls”)” untuk membersihkan visualisasi konsol sebelum memanggil fungsi “bannerTivaiz()” agar tampilan menu tetap rapi saat menerima *input* pilihan pengguna.

```
=====
|                                     |
|                                     TIVAIZ STORE                             |
|                                     |
|                                     |
|                                     |
|                                     INPUT BARANG                             |
|                                     |
|-----|
Nama Barang   : Buku
|
Stok (unit)   : 1
|
Harga (Rp)    : 10000
|
|-----|
[Berhasil] Buku ditambahkan! (Total barang: 1)
Press any key to continue . . . |
```

Gambar 4.13 Tampilan Sub-Menu 1 *Input* Barang.

Berdasarkan **Gambar 4.13** di atas, dapat diketahui bahwa ketika pengguna memilih opsi “1” pada sub-menu toko, program akan mengeksekusi fungsi “inputBarang()”. Fungsi ini diawali dengan pemeriksaan kondisi kapasitas penyimpanan untuk memastikan *array* belum penuh. Selanjutnya, pengguna diminta untuk memasukkan data spesifikasi barang yang meliputi pengisian *string* nama barang menggunakan perintah “cin.getline()”, serta pengisian data numerik berupa kuantitas stok dan nilai harga barang. Apabila seluruh rangkaian *input* data tersebut dinilai valid dan bebas dari kesalahan tipe data, program secara otomatis melakukan *increment* pada variabel penanda total, menyimpan data ke dalam *array*, serta menampilkan pesan konfirmasi keberhasilan penambahan item ke dalam sistem toko.

```
#-----#  
|                TIVAIZ STORE                |  
#-----#  
  
HAPUS BARANG  
  
-----  
No                  : 1  
Nama Barang        : Buku  
Stok Barang        : 1  
Harga/Stok         : Rp 10000  
  
-----  
Pilih nomor barang yang dihapus (1-1) : 1  
[Berhasil] Buku dihapus! (Total barang: 0)  
Press any key to continue . . .
```

Gambar 4.14 Tampilan Sub-Menu 2 Hapus Barang

Berdasarkan **Gambar 4.14** di atas, dapat diketahui bahwa ketika pengguna memilih opsi “2” pada sub-menu toko, alur kendali sistem akan memicu jalannya fungsi “hapusBarang()”. Fungsi ini pertama-tama akan memeriksa status ketersediaan barang; jika *array* tidak kosong, fungsi “tampilDaftar()” akan dipanggil untuk menampilkan seluruh rekapitulasi barang yang aktif. Pengguna kemudian diminta memasukkan nomor urut item target yang ingin disingkirkan dari daftar. Setelah nomor urut divalidasi dan dipastikan berada di dalam jangkauan batas *array*, program akan menjalankan logika penghapusan dengan metode menggeser seluruh elemen data di sisi kanan target ke arah kiri untuk menimpa memori lama, diikuti dengan pengurangan nilai variabel total agar basis data inventaris kembali sinkron.

Berdasarkan **Gambar 4.15** di atas, dapat diketahui bahwa ketika pengguna memilih opsi “3” pada sub-menu toko dan telah menentukan nomor urut barang valid yang ingin diubah, alur kendali sistem akan mengeksekusi logika internal di dalam fungsi “`editBarang()`”. Program terlebih dahulu mengonversi *input* nomor urut menjadi indeks *array* yang sesuai, kemudian menampilkan teks penunjuk Barang: Buku untuk memberikan konteks informasi item yang sedang aktif dimodifikasi. Pengguna selanjutnya disajikan sub-menu tingkat ketiga yang memuat beberapa opsi modifikasi spesifik, yaitu “1. Edit Harga” untuk memperbarui nilai nominal barang, “2. Tambah Stok” serta “3. Kurangi Stok” untuk memanipulasi kuantitas inventaris, dan “0. Kembali”. Melalui antarmuka perulangan *do-while* ini, *input* pilihan pengguna akan diteruskan untuk memicu fungsi aksi terkait yang menerima parameter secara *pass-by-reference*, sehingga setiap perubahan nilai yang terjadi di dalam sub-fungsi tersebut dapat langsung memperbarui basis data *array* utama secara *real-time*.

Berdasarkan **Gambar 4.16** di atas, dapat diketahui bahwa ketika pengguna memilih opsi “4” pada sub-menu toko, program akan mengeksekusi fungsi “lihatBarang()”. Fungsi ini melakukan pemeriksaan kondisi awal; jika data *array* tidak kosong, sistem akan menjalankan struktur perulangan *for* dari indeks 0 hingga total - 1 untuk melakukan rekapitulasi data. Program secara aktif mencetak informasi setiap item yang meliputi nomor urut, nama barang, sisa stok, dan harga per unit. Selain itu, program juga melakukan operasi matematika langsung berupa perkalian antara data stok dan harga “(stok[i] * harga[i])” untuk menghasilkan luaran kolom total harga per produk, serta merangkum akumulasi variasi produk di bagian bawah tampilan.

```

=====#
|                KALKULATOR                |
|=====#

1. Operasi Dasar (+, -, *, /)
2. Faktorial
0. Kembali

=====
Pilih menu: |

```

Gambar 4.17 Tampilan Sub-Menu Kalkulator TIVAIZ.

Berdasarkan **Gambar 4.17** di atas, dapat diketahui ketika pengguna memilih opsi “2” pada menu utama, alur kendali sistem akan memicu jalannya fungsi “menuKalkulator()”. Fungsi ini bertindak sebagai sub-pusat kendali antarmuka fitur matematika dengan membersihkan layar terlebih dahulu menggunakan perintah “system(“cls”)” dan memanggil “bannerKalkulator()”. Pengguna disajikan tiga opsi instruksi navigasi khusus, yaitu menu “1. Operasi Dasar (+, -, *, /)” untuk perhitungan aritmatika standar dua bilangan pecahan, “2. Faktorial” untuk memproses perhitungan bilangan bulat positif menggunakan metode fungsi rekursif, serta menu “0. Kembali” untuk mengalihkan kembali arah kendali program menuju fungsi menu utama.

```

=====#
|                KALKULATOR                |
|=====#

-----
Operator (+, -, *, /) :
+
-----
Angka pertama : 6
-----
Angka kedua   : 7
-----
6 + 7 = 13
Press any key to continue . . . |

```

Gambar 4.18 Tampilan Sub-Menu 1 Operasi Dasar.

Berdasarkan **Gambar 4.18** di atas, dapat diketahui bahwa ketika pengguna memilih opsi “1” pada sub-menu kalkulator, program akan mengalihkan alur kendali sistem untuk mengeksekusi fungsi “operasiDasar()”. Fungsi ini meminta pengguna untuk menentukan jenis operasi matematika dengan memasukkan karakter simbol aritmatika ke dalam variabel `ini_operator`. Jika karakter valid (seperti simbol +), pengguna diminta memasukkan dua nilai bilangan, yaitu variabel “angkaSatuy” dan “angkaDuaw”. Sistem kemudian mengeksekusi logika percabangan *if-else if* untuk mencocokkan operator dan melakukan perhitungan penjumlahan langsung. Fungsi ini juga dilengkapi pengaman komputasi untuk memeriksa kondisi pembagi khusus pada operasi pembagian (/) guna menghindari *error* pembagian dengan nol (*division by zero*).


```
#####
|          KALKULATOR          |
#####

-----
N : 7
-----
7! = 5040
Press any key to continue . . . |
```

Gambar 4.19 Tampilan Sub-Menu 2 Faktorial.

Berdasarkan **Gambar 4.19** di atas, dapat diketahui bahwa ketika pengguna memilih opsi “2” pada sub-menu kalkulator, alur kendali program akan meminta masukan sebuah bilangan bulat positif ke dalam variabel “*nakHitung_apa*”. Program terlebih dahulu menerapkan seleksi kondisi untuk memastikan nilai yang di-*input* tidak bernilai negatif. Jika kondisi tersebut terpenuhi, program akan memanggil fungsi “*faktorial()*” yang dirancang dengan nilai kembalian bertipe *long long* guna mencegah terjadinya kesalahan berupa *integer overflow*. Proses perhitungan pada fungsi ini menerapkan metode rekursif, di mana fungsi akan memanggil dirinya sendiri secara bertumpuk dengan rumus “ $n * \text{faktorial}(n - 1)$ ” hingga mencapai kondisi batas (*base case*) ketika nilai parameter sama dengan 0 untuk menghasilkan luaran nilai akumulasi perkalian yang final.

```
=====
Pilihan menu: 0
Terima kasih telah menggunakan TIVAIZ Store
=====
Process exited after 1238 seconds with return value 0
Press any key to continue . . . |
```

Gambar 4.20 Tampilan Menu Utama Pesan Perpisahan

Berdasarkan **Gambar 4.20** di atas, dapat diketahui bahwa ketika pengguna meng-*input* angka “0” pada perintah pemilihan menu utama, program akan menghentikan siklus perulangan *do-while* yang mengontrol jalannya fungsi “menuUtama()”. Alur logika percabangan kemudian akan mengeksekusi blok instruksi penutup untuk menampilkan pesan konfirmasi perpisahan berupa kalimat "Terima kasih telah menggunakan TIVAIZ Store" ke layar konsol. Setelah fungsi tersebut selesai dieksekusi, kendali program dikembalikan sepenuhnya ke fungsi “main()” yang langsung melempar nilai balik *return* 0, menandakan bahwa seluruh operasional program telah berhasil dihentikan secara normal dan aman oleh sistem tanpa adanya *error runtime (process exited with return value)*

4.5 ANALISIS PROGRAM

```
#include <iostream>
using namespace std;
const int MAX = 10;
const int MAX_namaBarang = 50;
```

Snippet di atas merupakan blok inisialisasi dan deklarasi konstanta global. Direktif “`#include <iostream>`” dan “`using namespace std;`” digunakan agar program bisa menjalankan fungsi `cin` dan `cout` tanpa perlu menulis prefiks “`std::`” di setiap baris. Komponen utama blok ini adalah dua konstanta global, yaitu “`MAX`” (nilai 10) dan “`MAX_namaBarang`” (nilai 50), yang berfungsi membatasi ukuran *array* inventaris secara terpusat. Nilai ini mengunci kapasitas maksimal toko (10 slot) dan panjang nama barang (50 karakter) untuk diakses oleh fungsi “`inputBarang()`”, “`hapusBarang()`”, “`editBarang()`”, hingga “`main()`”. Penempatan global ini membuat *maintenance* kode lebih efisien karena perubahan skala program cukup dilakukan dengan mengganti nilai di blok ini saja.

```
void menuUtama(char nama[][MAX_namaBarang], int stok[], int harga[], int
&total);
void menuTivaiz(char nama[][MAX_namaBarang], int stok[], int harga[], int
&total);

void inputBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int
&total);

void hapusBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int
&total);

void editBarang(char nama[][MAX_namaBarang], int stok[], int harga[],
int total);

void lihatBarang(char nama[][MAX_namaBarang], int stok[], int harga[],
int total);

void editHarga(int &harga, char namaBarang[]);
void tambahStok(int &stok, char namaBarang[]);
void kurangiStok(int &stok, char namaBarang[]);
void menuKalkulator();
void operasiDasar();
long long faktorial(int n);
```

Snippet di atas merupakan blok prototipe fungsi. Blok ini terdiri dari 11 fungsi berjenis *void* (tidak mengembalikan nilai) dan 1 fungsi berjenis *return function* bertipe data *long long* (faktorial). Beberapa fungsi menggunakan parameter biasa (*pass-by-value*), sementara fungsi modifikasi data menggunakan parameter referensi (*pass-by-reference*) bertanda `&`.

Fungsi-fungsi ini dideklarasikan di awal program agar *compiler* mengenali nama, tipe kembalian, dan argumen sebelum fungsi tersebut dijabarkan secara detail atau dipanggil oleh fungsi lain. Urgensi pembuatan prototipe ini adalah untuk mengatasi batasan urutan pembacaan kode C++ dari atas ke bawah (*top-down compilation*). Hal ini memungkinkan fungsi-fungsi di dalam program untuk saling memanggil secara bebas dan fleksibel (ruang lingkup pemanggilan global) tanpa memicu *error* kompilasi berupa '*identifier not found*'.

```
void bannerTivaiz (){

    cout << "#=====#" << endl;
    cout << "|                TIVAIZ STORE                |" << endl;
    cout << "#=====#" << endl <<
endl;
}
```

Snippet di atas merupakan blok fungsi “`bannerTivaiz()`”. Fungsi ini termasuk ke dalam jenis fungsi *void* (tanpa nilai kembalian) yang tidak menerima parameter atau argumen apa pun di dalam deklarasinya. Urgensi dari pembuatan fungsi ini adalah untuk menerapkan prinsip reusabilitas kode (*code reusability*) dan mencegah redudansi (pengulangan kode yang sia-sia). Daripada menuliskan rentetan perintah `cout` visualisasi *header* yang panjang secara berulang-ulang di setiap menu, baris kode tersebut diringkas ke dalam satu fungsi terisolasi. Alur fungsionalnya bertugas untuk mencetak visualisasi judul antarmuka toko ke layar konsol. Fungsi ini memiliki ruang lingkup pemanggilan yang fleksibel, di mana ia dipanggil secara berulang di dalam fungsi “`menuTivaiz()`”, “`inputBarang()`”, “`hapusBarang()`”, “`editBarang()`”, hingga “`lihatBarang()`” untuk menjaga konsistensi estetika tampilan program saat pengguna bernavigasi.

```
void bannerKalkulator (){

    cout << "#=====#" << endl;
    cout << "|                KALKULATOR                |" << endl;
    cout << "#=====#" << endl
<<endl;

}
```

Snippet di atas merupakan blok fungsi “`bannerKalkulator()`”. Fungsi ini termasuk ke dalam jenis fungsi *void* (tanpa nilai kembalian) yang tidak menerima parameter atau argumen apa pun di dalam deklarasinya. Urgensi dari pembuatan fungsi ini adalah untuk menerapkan prinsip reusabilitas kode (*code reusability*) guna menghindari penulisan redudan rentetan perintah `cout` dekorasi yang sama secara berulang-ulang. Alur

fungsionalnya bertugas untuk mencetak visualisasi *header* menu kalkulator ke layar konsol. Fungsi ini memiliki ruang lingkup pemanggilan lokal di dalam file, di mana ia dipanggil secara berulang pada fungsi “menuKalkulator()” dan fungsi “operasiDasar()” untuk menjaga kerapian estetika antarmuka program.

```
void tampilDaftar(char nama[][MAX_namaBarang], int stok[], int harga[], int
total) {

    for (int i = 0; i < total; i++) {

        cout << "No          : " << i + 1 << "\n";

        cout << "Nama Barang : " << nama[i] << "\n";

        cout << "Stok Barang : " << stok[i] << "\n";

        cout << "Harga/Stok  : Rp " << harga[i] << "\n";

        cout << "-----\n";

    }

}
```

Snippet di atas merupakan blok fungsi “tampilDaftar()”. Fungsi ini termasuk ke dalam jenis fungsi *void* (tanpa nilai kembalian) yang menerima parameter *array* multi-dimensi karakter *nama*, *array integer* *stok*, *array integer* *harga*, serta variabel *integer* *total* secara *pass-by-value* (hanya menyalin nilai karena tidak ada manipulasi data di dalamnya). Urgensi dari pembuatan fungsi ini adalah untuk menerapkan prinsip reusabilitas kode (*code reusability*) guna mencegah duplikasi perulangan cetak data yang sama pada fitur toko. Alur fungsional dan logikanya dikendalikan oleh perulangan *for* yang melakukan iterasi dari indeks 0 hingga total - 1 untuk membaca dan mencetak seluruh elemen data barang yang aktif secara berurutan. Fungsi ini memiliki ruang lingkup pemanggilan lokal, di mana ia dipanggil di dalam fungsi “hapusBarang()” dan “editBarang()” untuk memperlihatkan daftar barang sebelum pengguna memilih nomor target yang ingin dimodifikasi.

```

void menuUtama(char nama[][MAX_namaBarang], int stok[], int harga[], int
&total) {
    int pilihanUsertercinta;
    do{
        cout << "|=====|" << endl;
        cout << "|                                |" << endl;
        cout << "[                MODULE 4 - FUNCTION                ]" << endl;
        cout << "|                                |" << endl;
        cout << "|=====|" << endl <<
endl;
        cout << "1. TIVAIZ Store" << endl << endl;
        cout << "2. Kalkulator TIVAIZ" << endl << endl;
        cout << "0. Keluar" << endl << endl;
        cout << "===== " << endl;
        cout << "Pilihan menu: ";
        cin >> pilihanUsertercinta;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";
            pilihanUsertercinta = -1;
            system ("pause");
            system ("cls");
            continue;
        }
        if (pilihanUsertercinta == 1)
            menuTivaiz (nama, stok, harga, total);
        else if (pilihanUsertercinta == 2)
            menuKalkulator();
        else if (pilihanUsertercinta == 0)
            cout << "Terima kasih telah menggunakan TIVAIZ Store";
        else{
            cout << "[Gagal] Menu tidak valid! Pilih antara 0 sampai 2.\n";
            system ("pause");
            system ("cls");
        }
    }while (pilihanUsertercinta != 0);
}

}

cout << endl << endl;

```

Snippet di atas merupakan blok fungsi “menuUtama()”. Fungsi ini berjenis *void* (tanpa kembalian) yang menerima parameter *array* data toko serta variabel total secara referensi (&) agar perubahan nilai tersinkronisasi global. Urgensi pembuatan fungsi ini adalah sebagai pusat kendali navigasi teratas untuk memisahkan antarmuka menu dari fungsi “main()”,

sehingga struktur arsitektur kode menjadi modular dan rapi. Alur logikanya diatur oleh perulangan *do-while* yang mengunci menu selama “pilihanUsertercinta” tidak bernilai 0. Jika terjadi *error input* (bukan angka), blok “if (cin.fail())” akan mereset status *error* menggunakan “cin.clear()”, membuang *buffer* sisa menggunakan “cin.ignore()”, lalu mengulang menu menggunakan perintah *continue*. Jika *input* valid, seleksi kondisi *if-else if* akan menyalurkan program untuk memanggil fungsi “menuTivaiz()” atau “menuKalkulator()”. Fungsi ini memiliki ruang lingkup pemanggilan satu kali di dalam fungsi “main()” sebagai inisiator utama program.

```
void menuTivaiz (char nama[][MAX_namaBarang], int stok[], int harga[], int
    &total){
    int pilihanUsertercinta;
    do{
        system ("cls");

        bannerTivaiz();

        cout << "1. Input Barang\n" << endl;
        cout << "2. Hapus Barang\n" << endl;
        cout << "3. Edit Barang\n" << endl;
        cout << "4. Lihat Barang\n" << endl;
        cout << "0. Kembali\n" << endl;
        cout << "=====\n";
        cout << "Pilih menu: ";
        cin >> pilihanUsertercinta;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";
            pilihanUsertercinta = -1;

            system ("pause");

            system ("cls");

            continue;
        }
        if (pilihanUsertercinta== 1)
            inputBarang(nama, stok, harga, total);
        else if (pilihanUsertercinta == 2)
            hapusBarang(nama, stok, harga, total);
        else if (pilihanUsertercinta == 3)
            editBarang (nama, stok, harga, total);
        else if (pilihanUsertercinta == 4)
            lihatBarang(nama, stok, harga, total);
```

```

        else if (pilihanUsertercinta == 0) {
            system ("cls");
        }
        else {
            cout << "[Gagal] Menu tidak valid! Pilih antara 0 sampai 4.\n";
            system ("pause");
            system ("cls");
        }
    }while (pilihanUsertercinta != 0);

```

Snippet di atas merupakan fungsi “menuTivaiz()”. Fungsi ini berjenis *void* karena tidak mengembalikan nilai, serta menerima parameter berupa *array* data barang dan variabel total secara *reference* (&) agar modifikasi data di dalam sub-menu toko ini tetap sinkron pada memori utama. Urgensi dari pembuatan fungsi ini adalah sebagai sub-pusat kendali navigasi khusus untuk fitur manajemen inventaris TIVAIZ Store. Pemisahan ini bertujuan agar fitur operasional barang tidak menumpuk dan terpisah dari menu utama maupun fitur kalkulator. Alur logikanya dikendalikan oleh perulangan *do-while* yang mengunci tampilan sub-menu selama variabel “pilihanUsertercinta” tidak bernilai 0. Perintah “system(“cls”)” diletakkan di awal perulangan untuk membersihkan layar secara aktif agar visualisasi antarmuka tetap rapi. Jika terjadi kesalahan *input* berupa karakter non-angka, blok “if (cin.fail())” akan mengamankan aliran program dengan mereset status *error* menggunakan “cin.clear()” dan membuang sisa *buffer* lewat perintah “cin.ignore(1000, '\n')” sebelum mengulang siklus perulangan. Sebaliknya, jika *input* yang dimasukkan valid, struktur percabangan *if-else if* bertugas menyalurkan program untuk memanggil fungsi aksi yang sesuai, seperti “inputBarang()”, “hapusBarang()”, “editBarang()”, atau “lihatBarang()”. Fungsi “menuTivaiz()” ini nantinya dipanggil di dalam fungsi “menuUtama()” ketika pengguna memilih menu pertama.

```

void inputBarang (char nama[][MAX_namaBarang], int stok[], int harga[], int
&total){
    system ("cls");
    bannerTivaiz ();
    cout << "-----" << endl;
    cout << "                INPUT BARANG                " << endl <<
endl;
    cout << "-----" << endl;
    if (total >= MAX){
        cout << "[Gagal] Kapasitas barang penuh!\n";
        system ("pause");
        return;
    }
}

```

```

    cout << "Nama Barang   : ";
    cin.ignore();
    cin.getline(nama[total], MAX_namaBarang);
    cout << "-----\n";
    cout << "Stok (unit)   : ";
    cin >> stok[total];
    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Stok harus berupa angka!\n";
        system("pause");
        return;
    }
    cout << "-----\n";
    cout << "Harga (Rp)    : ";
    cin >> harga[total];
    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Harga harus berupa angka!\n";
        system("pause");
        return;
    }
    total++;
    cout << "\n[Berhasil] " << nama[total - 1]
        << " ditambahkan! (Total barang: " << total << ")\n";
    system("pause");
}

```

Snippet di atas merupakan fungsi “inputBarang()”. Fungsi ini berjenis *void* (tanpa kembalian) yang menerima parameter *array* data toko serta variabel *total* secara *reference* (&) untuk melakukan penulisan data item baru langsung ke memori *array* utama. Urgensi pembuatan fungsi ini adalah untuk menyediakan fitur penambahan *record* data inventaris baru secara dinamis, terisolasi dari fungsi kendali menu utama. Alur logikanya diawali seleksi kondisi “if (total >= MAX)” sebagai *guard clause* untuk membatasi *input* jika *array* sudah penuh. Karakter *string* dibaca menggunakan “cin.getline()” setelah “cin.ignore()” membersihkan *buffer*, sementara data numerik di-*input* ke indeks *array* senilai variabel *total*. Setiap *input* numerik divalidasi oleh “if (cin.fail())” guna mencegah *corrupt* data jika pengguna memasukkan karakter non-angka. Apabila semua validasi lolos, nilai *total* akan mengalami *increment* (“total++”) sebagai penanda berhasilnya penyimpanan data. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “menuTivaiz()”.


```

void hapusBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int
    &total) {
    system ("cls");
    bannerTivaiz();
    cout << "-----" << endl;
    cout << "                HAPUS BARANG                " << endl <<
    endl;
    cout << "-----" << endl;
    if (total == 0){
        cout << "[Gagal] Belum ada barang!\n";
        system ("pause");
        return;
    }
    tampilDaftar(nama, stok, harga, total);
    cout << "Pilih nomor barang yang dihapus (1-" << total << "): ";
    int pilihanUsertercinta;
    cin >> pilihanUsertercinta;
    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";
        system ("pause");
        return;
    }
    if (pilihanUsertercinta < 1 || pilihanUsertercinta > total) {
        cout << "[Gagal] Nomor " << pilihanUsertercinta << " tidak valid!"
            << " Pilih antara 1 sampai " << total << ".\n";
        system ("pause");
        return;
    }
    char namaHapus[MAX_namaBarang];
    for (int k = 0; k < MAX_namaBarang; k++) namaHapus[k] =
        nama[pilihanUsertercinta - 1][k];
    for (int i = pilihanUsertercinta - 1; i < total - 1; i++) {
        for (int k = 0; k < MAX_namaBarang; k++) nama[i][k] = nama[i + 1][k];
        stok[i] = stok[i + 1];
        harga[i] = harga[i + 1];
    }
    total--;
    cout << "[Berhasil] " << namaHapus
        << " dihapus! (Total barang: " << total << ")\n";
    system ("pause");
}

```

Snippet di atas merupakan fungsi “hapusBarang()”. Fungsi ini berjenis *void* (tanpa kembalian) yang menerima parameter data *array* toko serta variabel total secara *reference* (&) untuk memanipulasi pengurangan elemen data langsung dari memori global. Urgensi

pembuatan fungsi ini adalah untuk menyediakan fitur reduksi data atau eliminasi *record* barang tertentu dari sistem inventaris toko secara dinamis tanpa merusak susunan indeks *array*. Alur logikanya diawali dengan pengecekan kondisi *array* menggunakan “if (total == 0)”. Jika aman, fungsi “tampilDaftar()” dipanggil untuk menyajikan opsi target nomor barang. *Input* “pilihanUsertercinta” divalidasi dari tipe non-angka menggunakan “cin.fail()” serta batas jangkauan nilai indeksinya. Sebelum dihapus, *string* nama barang disalin sementara ke *array* “namaHapus” menggunakan perulangan elemen karakter demi keperluan cetak konfirmasi. Proses eliminasi data menerapkan metode *shifting*, di mana perulangan bersarang (*nested loop*) berjalan dari indeks target untuk menggeser elemen di sisi kanan ke arah kiri guna menimpa memori lama. Langkah ini ditutup dengan *decrement* nilai variabel total (“total–”) untuk menyinkronkan jumlah data. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “menuTivaiz()”.

```
void editHarga(int &harga, char namaBarang[]) {
    int hargaBaru;
    cout << "-----\n";
    cout << "Harga baru (Rp): ";
    cin >> hargaBaru;

    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";
        system("pause");
        return;
    }
    int hargaLama = harga;
    harga = hargaBaru;
    cout << "[Berhasil] Harga " << namaBarang
        << ": Rp " << hargaLama << " -> Rp " << harga << "\n";
    system("pause");
}
```

Snippet di atas merupakan fungsi “editHarga()”. Fungsi ini berjenis *void* (tanpa kembalian) yang menerima parameter data *integer* harga secara *reference* (&) serta *array* karakter “namaBarang” agar perubahan nilai harga baru dapat memanipulasi elemen *array* utama secara langsung. Urgensi pembuatan fungsi ini adalah untuk mengisolasi logika pembaruan nilai nominal komoditas ke dalam sub-fungsi spesifik, sehingga meringkas kompleksitas kode yang ada pada fungsi menu edit. Alur logikanya dimulai dengan meminta masukan variabel “hargaBaru” yang langsung divalidasi oleh blok seleksi kondisi “if (cin.fail())” guna menangani *error* tipe data non-angka. Jika *input* valid, nilai harga saat

ini disalin terlebih dahulu ke variabel lokal “hargaLama” demi kebutuhan visualisasi histori cetak. Selanjutnya, program melakukan operasi *assignment* di mana variabel harga langsung ditimpa oleh nilai “hargaBaru”. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “editBarang()” ketika pengguna memilih opsi operasional pertama.

```
void tambahStok(int &stok, char namaBarang[]) {
    int jumlah;
    cout << "-----\n";
    cout << "Jumlah tambah: ";
    cin >> jumlah;

    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";
        system("pause");
        return;
    }

    int stokLama = stok;
    stok += jumlah;
    cout << "[Berhasil] Stok " << namaBarang
        << ": " << stokLama << " -> " << stok << "\n";
    system("pause");
}
```

Snippet di atas merupakan fungsi “tambahStok()”. Fungsi ini berjenis *void* (tanpa nilai kembalian) yang menerima parameter data kuantitas barang secara *reference* (&) serta *array* karakter identitas produk agar manipulasi penjumlahan stok baru dapat langsung memperbarui data pada *array* utama. Urgensi pembuatan fungsi ini adalah untuk mengisolasi logika penambahan pasokan barang ke dalam sub-fungsi modular terpisah, sehingga meningkatkan keterbacaan struktur kode pada fungsi edit utama. Alur logikanya dimulai dengan mengambil *input* kuantitas pada variabel jumlah yang disaring oleh “if (cin.fail())” untuk menangkal *error* data non-angka. Jika validasi lolos, program menyalin nilai kuantitas saat ini ke variabel lokal “stokLama” sebagai data histori cetak konfirmasi. Selanjutnya, program mengeksekusi operasi *assignment* penjumlahan “(stok += jumlah)” untuk mengakumulasikan total komoditas secara *real-time*. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “editBarang()” saat mendeteksi pilihan opsi kedua dari pengguna.

```

void kurangiStok(int &stok, char namaBarang[]) {
    int jumlah;
    cout << "-----\n";
    cout << "Jumlah kurang: ";
    cin >> jumlah;

    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";
        system("pause");
        return;
    }
    if (jumlah >= stok) {
        cout << "[Gagal] Stok minimal 1!\n";
        cout << "Stok saat ini : " << stok << "\n";
        cout << "Maks dikurangi : " << stok - 1 << "\n";
        system("pause");
        return;
    }

    int stokLama = stok;
    stok -= jumlah;
    cout << "[Berhasil] Stok " << namaBarang
        << ": " << stokLama << " -> " << stok << "\n";
    system("pause");
}

}

}

cout << endl;
// cari index pertama dan terakhir yang bukan spasi
int awal = 0;
int akhir = panjangKalimat1 - 1;
while (kalimat1[awal] == ' ') awal++;
while (kalimat1[akhir] == ' ') akhir--;

```

Snippet di atas merupakan fungsi “kurangiStok()”. Fungsi ini berjenis *void* (tanpa nilai kembalian) yang menerima parameter kuantitas barang secara *reference* (&) serta *array* karakter nama produk agar operasi pengurangan nilai dapat langsung memanipulasi data pada *array* utama. Urgensi pembuatan fungsi ini adalah untuk mengisolasi logika reduksi atau pengurangan kuantitas barang ke dalam sub-fungsi modular terpisah, sehingga mencegah penumpukan baris kode pada fungsi edit utama. Alur logikanya diawali dengan mengambil *input* variabel jumlah yang divalidasi oleh blok “if (cin.fail())” untuk menyaring tipe data non-angka. Selanjutnya, program menerapkan filter pengondisian kedua melalui instruksi “if (jumlah >= stok)” sebagai aturan bisnis inventaris, guna memastikan

kuantitas akhir barang tidak habis total atau bernilai di bawah 1 unit. Jika kedua validasi tersebut lolos, nilai awal akan dicatat ke variabel lokal “stokLama”, diikuti dengan eksekusi operasi *assignment* pengurangan “(stok -= jumlah)” untuk memperbarui data komoditas secara *real-time*. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “editBarang()” saat mendeteksi pilihan opsi ketiga dari pengguna.

```
void editBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int
total) {
    system ("cls");
    bannerTivaiz();
    cout << "-----" << endl;
    cout << "                EDIT BARANG                " << endl <<
endl;
    cout << "-----" << endl;
    if (total == 0) {
        cout << "[Gagal] Belum ada barang!\n";
        system ("pause");
        return;
    }

    tampilDaftar(nama, stok, harga, total);

    cout << "Pilih nomor barang yang diedit (1-" << total << "): ";
    int pilihanUsertercinta;
    cin >> pilihanUsertercinta;

    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');
        cout << "[Gagal] Input harus berupa angka!\n";
        system ("pause");
        return;
    }

    if (pilihanUsertercinta < 1 || pilihanUsertercinta > total) {
        cout << "[Gagal] Nomor " << pilihanUsertercinta << " tidak valid!"
        << " Pilih antara 1 sampai " << total << ".\n";
        system ("pause");
        return;
    }

    int indeks = pilihanUsertercinta - 1;
    int opsi_dopsi;
    do {
        system ("cls");
        bannerTivaiz();
        cout << "-----" << endl;
        cout << "                EDIT BARANG                " << endl
<< endl;
        cout << "-----" << endl;
        cout << "Barang: " << nama[indeks] << "\n";
        cout << "-----\n";
        cout << "1. Edit Harga\n" << endl;
        cout << "2. Tambah Stok\n" << endl;
        cout << "3. Kurangi Stok\n" << endl;
        cout << "0. Kembali\n" << endl;
    } while (opsi_dopsi != 0);
}
```

```

        cout << "-----\n";
        cout << "Pilih opsi: ";
        cin >> opsi_dopsi;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";
            system ("pause");
            continue;
        }
        if (opsi_dopsi == 1)
            editHarga (harga[indeks], nama[indeks]);
        else if (opsi_dopsi == 2)
            tambahStok (stok[indeks], nama[indeks]);
        else if (opsi_dopsi == 3)
            kurangiStok(stok[indeks], nama[indeks]);
        else if (opsi_dopsi == 0) {}
        else {
            cout << "[Gagal] Opsi tidak valid!\n";
            system ("pause");
        }
    } while (opsi_dopsi != 0);
}

```

Snippet di atas merupakan fungsi “`editBarang()`”. Fungsi ini berjenis *void* (tanpa kembalian) yang menerima parameter *array* data toko serta variabel total secara nilai (*pass-by-value*), karena fungsi ini bertugas menjembatani modifikasi elemen *array* yang ditunjuk tanpa mengubah struktur kapasitas atau jumlah *record* total itu sendiri. Urgensi pembuatan fungsi ini adalah sebagai sub-kendali atau perantara khusus operasional modifikasi elemen barang, yang mengisolasi menu pilihan edit agar tidak menumpuk di fungsi “`menuTivaiz()`”. Alur logikanya dimulai dengan *guard clause* “`if (total == 0)`” untuk memastikan *array* berisi data. Jika aman, fungsi “`tampilDaftar()`” dipanggil untuk memperlihatkan opsi target. *Input* nomor urut divalidasi keandalannya, lalu dikonversi menjadi indeks *array* (“`pilihanUsertercinta - 1`”). Program kemudian memasuki siklus perulangan *do-while* yang menyajikan antarmuka sub-menu tingkat ketiga. Di dalam *loop* ini, *input* variabel “`opsi_dopsi`” disaring dari tipe non-angka menggunakan “`cin.fail()`”. Aliran data kemudian disalurkan melalui struktur *if-else if* untuk memanggil fungsi aksi spesifik, yaitu “`editHarga()`”, “`tambahStok()`”, atau “`kurangiStok()`” dengan mengirimkan parameter *array* target secara referensi alamat memori agar dapat diubah secara *real-time*. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “`menuTivaiz()`”.

```

void lihatBarang(char nama[][MAX_namaBarang], int stok[], int harga[], int
total) {
    system ("cls");
    bannerTivaiz();
    cout << "-----" << endl;
    cout << "                LIHAT BARANG                " << endl
<< endl;
    cout << "-----" << endl;
    if (total == 0) {
        cout << "Belum ada barang di toko.\n";
    } else {
        for (int i = 0; i < total; i++) {
            cout << "No          : " << i + 1 << "\n";
            cout << "Nama Barang : " << nama[i] << "\n";
            cout << "Stok Barang : " << stok[i] << "\n";
            cout << "Harga/Stok  : Rp " << harga[i] << "\n";
            cout << "Total Harga : Rp " << stok[i] * harga[i] << "\n";
            cout << "-----\n";
        }
        cout << "Total barang : " << total << " jenis\n";
    }

    cout << "-----\n";
    system ("pause");
}

```

Snippet di atas merupakan fungsi “lihatBarang()”. Fungsi ini berjenis *void* (tanpa kembalian) yang menerima parameter data *array* toko serta variabel total secara nilai (*pass-by-value*), karena fungsi ini hanya berfokus pada visualisasi pembacaan data tanpa melakukan mutasi pada nilai asli *array*. Urgensi pembuatan fungsi ini adalah untuk menyediakan menu rekapitulasi data inventaris toko yang komprehensif secara modular, terisolasi dari fungsi operasional data lainnya. Alur logikanya dikendalikan struktur percabangan “if (total == 0)” untuk mendeteksi apakah toko masih kosong. Jika terisi, program mengeksekusi blok *else* yang memuat perulangan *for* dari indeks 0 hingga total - 1 untuk mencetak seluruh informasi barang. Pada setiap iterasi, program secara langsung melakukan operasi perkalian aritmatika “(stok[i] * harga[i])” untuk menghitung nilai total aset per jenis barang. Siklus ini ditutup dengan menampilkan rangkuman akumulasi jenis barang bawaan variabel total. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “menuTivaiz()”.

```

void menuKalkulator() {
    int pilihanUsertercinta;
    do {
        system ("cls");
        bannerKalkulator();
        cout << "1. Operasi Dasar (+, -, *, /)\n" << endl;
        cout << "2. Faktorial\n" << endl;
        cout << "0. Kembali\n" << endl;
        cout << "=====\n";
        cout << "Pilih menu: ";
        cin >> pilihanUsertercinta;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "[Gagal] Input harus berupa angka!\n";
            pilihanUsertercinta = -1;
            system ("pause");
            system ("cls");
            continue;
        }

        if (pilihanUsertercinta == 1) {
            operasiDasar();
        } else if (pilihanUsertercinta == 2) {
            system ("cls");
            bannerKalkulator();
            cout << "-----\n";
            int nakHitung_apa;
            cout << "N : ";
            cin >> nakHitung_apa;

            if (cin.fail()) {
                cin.clear();
                cin.ignore(1000, '\n');
                cout << "[Gagal] Input harus berupa angka!\n";
                system ("pause");
                continue;
            }

            cout << "-----\n";
            if (nakHitung_apa < 0) {
                cout << "[Gagal] Faktorial tidak bisa untuk angka
negatif!\n";
            } else {
                cout << nakHitung_apa << "! = " << faktorial(nakHitung_apa)
<< "\n";
            }
            system ("pause");
        } else if (pilihanUsertercinta == 0) { //balik
            system ("cls");
        } else {
            cout << "[Gagal] Menu tidak valid! Pilih antara 0 sampai 2.\n";
            system ("pause");
            system ("cls");
        }

    } while (pilihanUsertercinta != 0);
}

```


Snippet di atas merupakan fungsi “menuKalkulator()”. Fungsi ini berjenis *void* (tanpa nilai kembalian) yang tidak menerima parameter apa pun karena operasional variabel perhitungan dilakukan secara internal dalam cakupan blok fungsi tersebut. Urgensi pembuatan fungsi ini adalah sebagai sub-pusat kendali antarmuka khusus fitur perhitungan matematika, guna memisahkan interaksi kalkulator dari menu utama toko sehingga arsitektur kode tetap modular. Alur logikanya dikendalikan oleh struktur perulangan *do-while* yang mengunci layar selama “pilihanUsertercinta” tidak bernilai 0. Fungsi ini memuat dua filter validasi “cin.fail()” untuk menangani *error input* non-angka pada pemilihan menu maupun saat memasukkan nilai *integer* “nakHitung_apa”. Aliran instruksi kemudian disalurkan melalui seleksi kondisi *if-else if*; opsi 1 mengarahkan program ke fungsi “operasiDasar()”, sedangkan opsi 2 meminta nilai untuk dieksekusi oleh fungsi *return* rekursif faktorial() setelah dipastikan nilainya tidak negatif “(nakHitung_apa < 0)”. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “menuUtama()”.

```
void operasiDasar() {
    system ("cls");
    bannerKalkulator();
    cout << "-----\n";

    char ini_operator;
    cout << "Operator (+, -, *, /) : ";
    cin >> ini_operator;

    if (ini_operator != '+' && ini_operator != '-' && ini_operator != '*' &&
    ini_operator != '/') {

        cout << "[Gagal] Operator tidak valid!\n";
        system ("pause");
        return;
    }

    float angkaSatuy, angkaDuaw;
    cout << "-----\n";
    cout << "Angka pertama : ";
    cin >> angkaSatuy;

    if (cin.fail()) {
        cin.clear();
        cin.ignore(1000, '\n');

        cout << "[Gagal] Input harus berupa angka!\n";
        system ("pause");
        return;
    }
    cout << "-----\n";
    cout << "Angka kedua : ";
    cin >> angkaDuaw;

    if (cin.fail()) {
        cin.clear();
```

```

        cin.ignore(1000, '\n');

        cout << "[Gagal] Input harus berupa angka!\n";
        system("pause");
        return;
    }

    cout << "-----\n";
    if      (ini_operator == '+') cout << angkaSatuy << " + " << angkaDuaw <<
    " = " << angkaSatuy + angkaDuaw << "\n";

    else if (ini_operator == '-') cout << angkaSatuy << " - " << angkaDuaw <<
    " = " << angkaSatuy - angkaDuaw << "\n";

    else if (ini_operator == '*') cout << angkaSatuy << " * " << angkaDuaw <<
    " = " << angkaSatuy * angkaDuaw << "\n";

    else if (ini_operator == '/') {
        if (angkaDuaw == 0) cout << "[Gagal] Tidak bisa dibagi nol!\n";
        else                cout << angkaSatuy << " / " << angkaDuaw << " = " <<
        angkaSatuy / angkaDuaw << "\n";
    }

    system("pause");
    system("cls");

```

Snippet di atas merupakan fungsi “operasiDasar()”. Fungsi ini berjenis *void* (tanpa kembalian) yang tidak menerima parameter *input* apa pun karena seluruh proses instansiasi dan pengolahan data aritmatika dilakukan di dalam ruang lingkup internal fungsi. Urgensi pembuatan fungsi ini adalah untuk mengisolasi baris kode perhitungan aritmatika standar (penjumlahan, pengurangan, perkalian, pembagian) agar terpisah dari struktur sub-menu utama kalkulator, sehingga sistem menjadi lebih terstruktur. Alur logikanya dimulai dengan mengambil *input* karakter ke variabel `ini_operator`, yang langsung divalidasi oleh filter kondisi gabungan logika AND (&&) untuk menolak karakter di luar simbol standar. Program kemudian membaca data operand menggunakan tipe data *float* pada variabel “angkaSatuy” dan “angkaDuaw”, lengkap dengan pengaman “`cin.fail()`” untuk menangkal *input* non-angka. Aliran kendali eksekusi diakhiri dengan struktur percabangan *if-else if* untuk menghitung hasil operasi matematika. Khusus pada opsi pembagian (/), sistem menerapkan seleksi bersarang (*nested if*) guna memeriksa kondisi nilai pembagi “(`angkaDuaw == 0`)” demi mencegah terjadinya *error fatal runtime* akibat *division by zero*. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “`menuKalkulator()`”.

```
long long faktorial(int n) {
    if (n == 0) return 1;
    return n * faktorial(n - 1);
}
```

Snippet di atas merupakan fungsi “faktorial()”. Berbeda dengan fungsi sebelumnya, fungsi ini termasuk ke dalam jenis fungsi yang mengembalikan nilai (*return function*) dengan tipe data *long long*, guna menampung hasil akumulasi perkalian numerik skala besar agar terhindar dari *error integer overflow*. Fungsi ini menerima satu parameter *integer* *n* secara *pass-by-value*. Urgensi pembuatan fungsi ini adalah untuk menerapkan metode rekursi (fungsi yang memanggil dirinya sendiri) guna menyelesaikan perhitungan matematika faktorial secara ringkas dan efisien tanpa menggunakan perulangan iteratif biasa (*looping*). Alur logikanya dikendalikan oleh kondisi basis (*base case*) berupa instruksi “if (n == 0)” yang akan menghentikan rekursi dan langsung mengembalikan nilai 1 ketika variabel “n” telah bernilai nol. Jika kondisi basis belum terpenuhi, fungsi mengeksekusi langkah rekursif (*recurrence relation*) dengan mengembalikan hasil operasi perkalian “n” dikalikan hasil dari pemanggilan ulang fungsi itu sendiri dengan argumen yang didekrementasikan “(n - 1)”. Proses pemanggilan bertumpuk ini akan terus berjalan mundur hingga menyentuh *base case*. Fungsi ini berada dalam ruang lingkup pemanggilan lokal di dalam fungsi “menuKalkulator()”.

```
int main (){
    char namaBarang[MAX][MAX_namaBarang];
    int stokBarang[MAX];
    int hargaBarang[MAX];
    int totalBarang = 0;
    menuUtama(namaBarang, stokBarang, hargaBarang, totalBarang);
    return 0;
}
```

Snippet di atas merupakan fungsi “main()”. Fungsi ini bertindak sebagai fungsi utama (*entry point*) atau titik awal eksekusi program C++ oleh sistem operasi. Fungsi ini mengembalikan nilai *integer* 0 (*return 0*) ke sistem sebagai sinyal bahwa seluruh rangkaian program telah berhasil dieksekusi tanpa *error*. Urgensi pembuatan fungsi ini adalah sebagai wadah inisialisasi variabel memori utama data toko dan sekaligus bertindak sebagai pemicu utama (inisiator) berjalannya program secara keseluruhan. Alur logikanya diawali dengan

deklarasi variabel lokal untuk menampung basis data inventaris, meliputi matriks *array* dua dimensi “namaBarang”, *array* satu dimensi “stokBarang” dan “hargaBarang” yang dimensinya dikunci oleh konstanta global “MAX”, serta variabel “totalBarang” yang langsung diinisialisasi dengan nilai awal 0. Setelah seluruh alokasi memori lokal terbuat, program langsung memanggil fungsi “menuUtama()” dengan melemparkan variabel-variabel tersebut sebagai argumen. Langkah pemanggilan ini secara otomatis mengalihkan kendali eksekusi program ke dalam modul navigasi utama hingga pengguna memilih opsi keluar. Fungsi ini berada dalam ruang lingkup tertinggi eksekusi dan tidak dipanggil oleh fungsi lain di dalam kode program.

4.6 KESIMPULAN

Berdasarkan hasil praktikum pada Modul IV tentang *Function*, dapat diambil kesimpulan bahwa:

1. Fungsi (*function*) dalam bahasa C++ adalah blok kode yang dirancang untuk menjalankan tugas tertentu dan dapat dipanggil berkali-kali dalam program. Fungsi memiliki komponen utama berupa nama fungsi, tipe kembalian (*return type*), parameter, dan isi fungsi (*body*). Dengan memahami struktur fungsi, mahasiswa dapat memisahkan logika program menjadi bagian-bagian yang lebih terorganisir dan mudah dipahami.
2. Jenis-jenis fungsi dalam C++ dapat diimplementasikan sesuai kebutuhan program, meliputi fungsi tanpa parameter dan tanpa nilai kembalian (*void*), fungsi dengan parameter, fungsi dengan nilai kembalian, serta fungsi rekursif. Setiap jenis fungsi memiliki karakteristik dan kegunaan yang berbeda, sehingga pemilihan jenis fungsi yang tepat sangat menentukan efisiensi dan keterbacaan program yang dibuat.
3. Penggunaan fungsi dalam program bertujuan untuk menerapkan prinsip *modularitas*, yaitu memecah program besar menjadi modul-modul kecil yang masing-masing memiliki tugas spesifik. Selain menghindari penulisan kode yang berulang (*redundancy*), fungsi juga mempermudah proses *debugging*, pemeliharaan, serta pengembangan program secara keseluruhan sehingga program menjadi lebih terstruktur dan efisien.

DAFTAR PUSTAKA

- [1] T. H. Harumy, H. F. Febriana, *et al.*, *Belajar Dasar Algoritma & Pemrograman C++*, 2016.
- [2] A. D. Samala, B. R. Fajri, and F. Ranuharja, *Pemrograman C++: From Zero to One*, Padang: UNP Press, 2021.
- [3] B. Raharjo, *Pemrograman C++ Revisi Kedua: Mudah dan Cepat Menjadi Master C++*, Bandung: Informatika, 2018.
- [4] E. Winarno dan Z. A. Hasibuan, *Pemrograman C++ untuk Pemula*, Jakarta: Elex Media Komputindo, 2016.